

Seite 1

Beispiel 12: $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \varepsilon$

1) Startvariable

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

2) ε -Regel

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \varepsilon \mid S$
 $B \rightarrow b$

3) Unit-Regel

$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln

$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $A_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

ABLEITUNGSDREIECK

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, von denen das Teilwort abgeleitet werden kann.

Beispiel

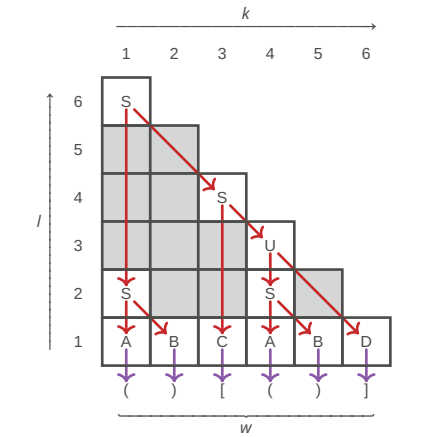
Parsen des Wortes $()[(())]$

$S \rightarrow AB \mid CD \mid CU \mid SS$
 $U \rightarrow SD$
 $A \rightarrow ($

$B \rightarrow)$
 $C \rightarrow [$
 $D \rightarrow]$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heißen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .



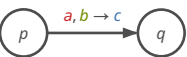
BNF

- Variablen: **<variablen-name>**
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

STACKAUTOMAT (PDA)

$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q: Zustände
- Σ : Eingabe-Alphabet
- Γ : Stack-Alphabet
- $\delta: Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$
- $q_0 \in Q$: Startzustand
- $F \subset Q$: Akzeptierzustände



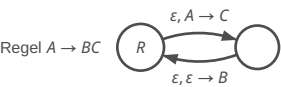
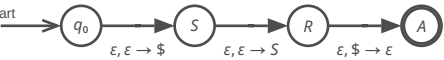
a vom Input

b vom Stack entfernen (Bedingung)

c auf den Stack

GRAMMATIK ABLESEN

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P, der L akzeptiert, $L = L(P)$.



TURING MASCHINEN
(deterministische) Turing-Maschine
 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

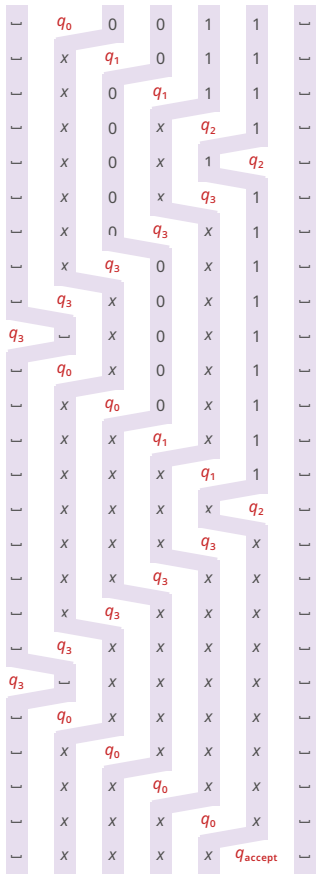
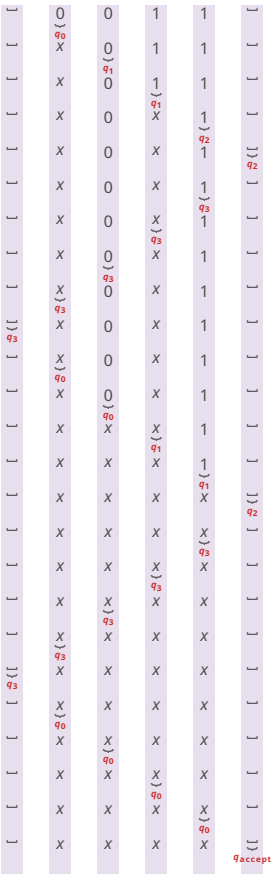
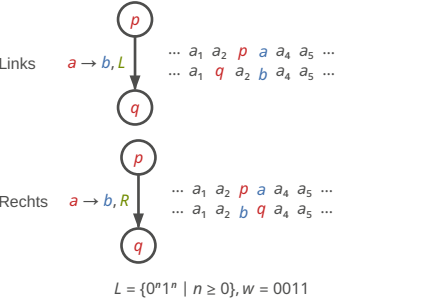
- Q: Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $_ \in \Gamma \setminus \Sigma$
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

$\delta(p, a) = (q, b, L)$:

- Übergang möglich, wenn a unter dem Schreib- / Lese-Kopf
- Aktuelles Feld auf dem Band wird mit b überschrieben
- Kopfbewegung: L links, R rechts

PRÜFUNG
Bei der Letzten Aufgabe (Welche Sprache wird von M akzeptiert?) muss eine ausformulierte Begründung stehen.

BERECHNUNGSGESCHICHTE



VARIANTEN

NICHTDETERMINISTISCHE TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten:
 $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

Simulation auf Standard-TM

Verwende 3 Bänder:

- Arbeitsband
- Kopie von w
- Liste aller Folgen von Wahlmöglichkeiten

ENTSCHEIDBARE PROBLEME

LEERHEITSPROBLEME

✓ Entscheidbar

$$E_{\text{DEA}} = \{\langle A \rangle \mid L(A) = \emptyset\}$$

Wie: Minimalautomat hat keinen Akzeptierzustand

✓ Entscheidbar

$$E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$$

Wie: Chomsky-Normalform

✗ Entscheidbar

$$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

Denn: Reduktion möglich: $A_{\text{TM}} \leq E_{\text{TM}}$

GLEICHEITSPROBLEME

✓ Entscheidbar

$$\text{EQ}_{\text{DEA}} = \{\langle A_1, A_2 \rangle \mid L(A_1) = L(A_2)\}$$

Wie: Vergleich der minimalen Automaten, oder Leerheitsproblem bei: $L(A_1) \triangle L(A_2)$

✗ Entscheidbar

$$\text{EQ}_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}$$

Denn: Es gibt eine Reduktion $\text{ALL}_{\text{CFG}} \leq \text{EQ}_{\text{CFG}}$. Da ALL_{CFG} nicht entscheidbar, auch EQ_{CFG} nicht entscheidbar.

✗ Entscheidbar

$$\text{EQ}_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$$

Denn: EQ_{PDA} nicht entscheidbar

VERSCHIEDENE BANDALPHABETE

Jedes andere Alphabet kann binär codiert werden.

Simulierbar in Zeit $O(t(n))$

MEHRSPURMASCHINE

Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

Die unabhängige Bewegung der Schreib-/Leseköpfe auf einer n -Band Maschine kann mithilfe eines $2n$ -spurigen Bandes (Daten + Zeiger für Schreib-/Lesekopf) nachgebildet werden.

Simulierbar in Zeit $O(t(n)^2)$

EINSEITIG UNENDLICHES BAND

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

AUFZÄHLER

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann Turing-erkennbar.

AKZEPTANZPROBLEME

✓ Entscheidbar

$$A_{\text{DEA}} = \{\langle A, w \rangle \mid w \in L(A)\}$$

Wie: Regex-Engines simulieren beliebige DEAs auf beliebigen Input-Wörtern w

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid w \in L(G)\}$$

Wie: CYK, deterministischer Parse-Algorithmus

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid \varepsilon \in L(G)\}$$

Wie: wenn CNF Regel $S \rightarrow \varepsilon$ enthält

✗ Entscheidbar

$$A_{\text{TM}} = \{\langle M, w \rangle \mid w \in L(M)\}$$

Denn: Halteproblem

ALLE WÖRTER PRODUZIEREN

✗ Entscheidbar

$$\text{ALL}_{\text{CFG}} = \{\langle G \rangle \mid G \text{ produziert } \Sigma^*\}$$

Denn: (Gegenteil von E Problem mit TM) $A_{\text{TM}} \leq \text{ALL}_{\text{PDA}}$

✗ Entscheidbar

$$\text{ALL}_{\text{PDA}} = \{\langle P \rangle \mid P \text{ akzeptiert } \Sigma^*\}$$

Denn: Reduktionsabbildung der Gegenteil akzeptiert

WEITERE

✗ Entscheidbar

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$$

Denn: Auf A_{TM} reduziert $\rightarrow$ nicht entscheidbar

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

NP

Eine Sprache L gehört zur Klasse NP , wenn L von einer **nicht**deterministischen TM in **polynomieller Zeit** **entschieden** werden kann

POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$$w \in L \Leftrightarrow V \text{ akzeptiert } \langle w, c \rangle$$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

Bei der Prüfung **Entscheidbarkeit** und **Zertifikat** erwähnen!

ENTSCHEIDBARE PROBLEME

LEERHEITSPROBLEME

✓ Entscheidbar

$$E_{\text{DEA}} = \{\langle A \rangle \mid L(A) = \emptyset\}$$

Wie: Minimalautomat hat keinen Akzeptierzustand

✓ Entscheidbar

$$E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$$

Wie: Chomsky-Normalform

✗ Entscheidbar

$$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

Denn: Reduktion möglich: $A_{\text{TM}} \leq E_{\text{TM}}$

GLEICHEITSPROBLEME

✓ Entscheidbar

$$\text{EQ}_{\text{DEA}} = \{\langle A_1, A_2 \rangle \mid L(A_1) = L(A_2)\}$$

Wie: Vergleich der minimalen Automaten, oder Leerheitsproblem bei: $L(A_1) \triangle L(A_2)$

✗ Entscheidbar

$$\text{EQ}_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}$$

Denn: Es gibt eine Reduktion $\text{ALL}_{\text{CFG}} \leq \text{EQ}_{\text{CFG}}$. Da ALL_{CFG} nicht entscheidbar, auch EQ_{CFG} nicht entscheidbar.

✗ Entscheidbar

$$\text{EQ}_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$$

Denn: EQ_{PDA} nicht entscheidbar

AKZEPTANZPROBLEME

✓ Entscheidbar

$$A_{\text{DEA}} = \{\langle A, w \rangle \mid w \in L(A)\}$$

Wie: Regex-Engines simulieren beliebige DEAs auf beliebigen Input-Wörtern w

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid w \in L(G)\}$$

Wie: CYK, deterministischer Parse-Algorithmus

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid \varepsilon \in L(G)\}$$

Wie: wenn CNF Regel $S \rightarrow \varepsilon$ enthält

✗ Entscheidbar

$$A_{\text{TM}} = \{\langle M, w \rangle \mid w \in L(M)\}$$

Denn: Halteproblem

ALLE WÖRTER PRODUZIEREN

✗ Entscheidbar

$$\text{ALL}_{\text{CFG}} = \{\langle G \rangle \mid G \text{ produziert } \Sigma^*\}$$

Denn: (Gegenteil von E Problem mit TM) $A_{\text{TM}} \leq \text{ALL}_{\text{PDA}}$

✗ Entscheidbar

$$\text{ALL}_{\text{PDA}} = \{\langle P \rangle \mid P \text{ akzeptiert } \Sigma^*\}$$

Denn: Reduktionsabbildung der Gegenteil akzeptiert

WEITERE

✗ Entscheidbar

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$$

Denn: Auf A_{TM} reduziert $\rightarrow$ nicht entscheidbar

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

LÖSUNGSANLEITUNG EINER PRÜFUNGSFRAGE:

– Nichttriviale Eigenschaft P aufschreiben

– Die beiden Sprachen $L(M_1)$ und $L(M_2)$ bilden (meistens kann man die leere Menge oder Σ^* als eine dieser Sprachen verwenden)

– Gibt es ein Programm, welches beide Sprachen erkennen kann? Sind beide Sprachen Turing erkennbar?

– Dann besagt der Satz von Rice, dass die Sprache nicht entscheidbar ist

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Anwendung des Satzes von Rice (R) 2 Punkte.

NP

Eine Sprache L gehört zur Klasse NP , wenn L von einer **nicht**deterministischen TM in **polynomieller Zeit** **entschieden** werden kann

POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$$w \in L \Leftrightarrow V \text{ akzeptiert } \langle w, c \rangle$$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

Bei der Prüfung **Entscheidbarkeit** und **Zertifikat** erwähnen!

ENTSCHEIDBARE PROBLEME

LEERHEITSPROBLEME

✓ Entscheidbar

$$E_{\text{DEA}} = \{\langle A \rangle \mid L(A) = \emptyset\}$$

Wie: Minimalautomat hat keinen Akzeptierzustand

✓ Entscheidbar

$$E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$$

Wie: Chomsky-Normalform

✗ Entscheidbar

$$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

Denn: Reduktion möglich: $A_{\text{TM}} \leq E_{\text{TM}}$

GLEICHEITSPROBLEME

✓ Entscheidbar

$$\text{EQ}_{\text{DEA}} = \{\langle A_1, A_2 \rangle \mid L(A_1) = L(A_2)\}$$

Wie: Vergleich der minimalen Automaten, oder Leerheitsproblem bei: $L(A_1) \triangle L(A_2)$

✗ Entscheidbar

$$\text{EQ}_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}$$

Denn: Es gibt eine Reduktion $\text{ALL}_{\text{CFG}} \leq \text{EQ}_{\text{CFG}}$. Da ALL_{CFG} nicht entscheidbar, auch EQ_{CFG} nicht entscheidbar.

✗ Entscheidbar

$$\text{EQ}_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$$

Denn: EQ_{PDA} nicht entscheidbar

AKZEPTANZPROBLEME

✓ Entscheidbar

$$A_{\text{DEA}} = \{\langle A, w \rangle \mid w \in L(A)\}$$

Wie: Regex-Engines simulieren beliebige DEAs auf beliebigen Input-Wörtern w

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid w \in L(G)\}$$

Wie: CYK, deterministischer Parse-Algorithmus

✓ Entscheidbar

$$A_{\text{CFG}} = \{\langle G, w \rangle \mid \varepsilon \in L(G)\}$$

Wie: wenn CNF Regel $S \rightarrow \varepsilon$ enthält

✗ Entscheidbar

$$A_{\text{TM}} = \{\langle M, w \rangle \mid w \in L(M)\}$$

Denn: Halteproblem

ALLE WÖRTER PRODUZIEREN

✗ Entscheidbar

$$\text{ALL}_{\text{CFG}} = \{\langle G \rangle \mid G \text{ produziert } \Sigma^*\}$$

Denn: (Gegenteil von E Problem mit TM) $A_{\text{TM}} \leq \text{ALL}_{\text{PDA}}$

✗ Entscheidbar

$$\text{ALL}_{\text{PDA}} = \{\langle P \rangle \mid P \text{ akzeptiert } \Sigma^*\}$$

Denn: Reduktionsabbildung der Gegenteil akzeptiert

WEITERE

✗ Entscheidbar

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$$

Denn: Auf A_{TM} reduziert $\rightarrow$ nicht entscheidbar

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

Schritt	Aufwand
1 Sudoku-Verifizierer	$O(n^6)$
2 Für jedes Feld überprüfen, ob die Summe der Zahlen im gleichen Käfig eine Quadratzahl ist.	$O(n^4 \cdot n^4)$
Total	$O(n^8)$

Entscheidbarkeit (E) 1 Punkt, Verifizierer (V) 1 Punkt, Zertifikat (Z) 1 Punkt, bestehende Sudoku Regeln sind in polynomieller Zeit verifizierbar (S) 1 Punkt, Verifikation von zwei zusätzlichen Regel (R) 1 Punkt, Laufzeit polynomiell (L) 1 Punkt.

m-SUDOKU

Entscheidbar? Ja: $n = m^4$ = Anzahl Felder
Zertifikat: Vollständig ausgefülltes Sudoku
Vorgehen:

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

Schritt	Aufwand
1 Jedes Feld: Zeichen kommt auf der Zeile nicht mehr vor	$O(n^4 \cdot n^2)$
2 Jedes Feld: Zeichen kommt in der Spalte nicht mehr vor	$O(n^4 \cdot n^2)$
3 Jedes Feld: Zeichen kommt im Unterquadrat nicht vor	$O(n^4 \cdot n^2)$
Total	$O(n^6)$

NURIKABE

Entscheidbar? Ja: $n = uv$ = Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.
Zertifikat: Liste der schwarzen Felder
Vorgehen:

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Anwendung des Satzes von Rice (R) 2 Punkte.

Schritt	Aufwand
1 Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^2)$
2 Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3 Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4 Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5 Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
Total	$O(n^3)$

DAMEN

Das n -Damenproblem ist die Aufgabe, eine Platzierung von n Damen auf einem $n \times n$ -Schachfeld zu finden, so dass sich die Damen nicht gegenseitig schlagen können.

Entscheidbar? Ja, es kann ein polynomieller Verifizierer erstellt werden.

Zertifikat: Das Wort $(n, r_1, r_2, \dots, r_n) \in \text{QUEENS}$, wobei r_i die Zeilennummer der Dame in Spalte i ist.

Vorgehen:

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

Schritt	Aufwand
1 Enthält das Wort genau $n + 1$ natürliche Zahlen zwischen 1 und n ?	$O(n \log(n))$
2 Sind die Zahlen r_i alle verschieden?	$O(n^2 \log(n)^2)$
3 Sind keine Damen in der gleichen Diagonalen platziert?	$O(n^2 \log(n)^2)$
Total	$O(n^2 \log(n)^2)$

Die Faktoren $\log(n)$ kommen daher, dass mit n auch die Länge der Zahlen wie $\log(n)$ anwächst. Entsprechend nimmt auch die Zeit zu, jede dieser Zahlen zu verarbeiten. Die gesamte Laufzeit ist schneller als $O(n^3)$, also polynomiell. Damit ist der oben beschriebene Algorithmus ein polynomieller Verifizierer.

REDUKTION

Polynomielle Reduktion: $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

TURING-VOLLSTÄNDIG

Eine Sprache A heisst eine **Programmiersprache**, wenn es eine Abbildung $c : A \mapsto \Sigma^*$ gibt, wobei $c(w)$ ein Programm für eine universelle Turing-Maschine ist.

Eine Programmiersprache heisst **Turing-vollständig**, wenn in ihr jede beliebige Turing-Maschine simuliert werden kann.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Beispiel 16: Turing-Entscheidbarkeit

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Beispiel 17: Turing-Entscheidbarkeit

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

KARP-KATALOG

PROBLEME MIT k ZAHLEN

VERTEX-COLORING

EXACT-CLIQUE-COVER

VERTEX-COVER

SET-COVERING

STEINER-TREE

FEEDBACK-NODE-SET

GRAPH-ÜBERDECKUNGEN

VERTEX-COLORING

CLIQUE-COVER

EXACT-CLIQUE-COVER

K-CLIQUE

VERTEX-COVERING

CLIQUE-COVER

EXACT-CLIQUE-COVER

K-CLIQUE

VERTEX-COLORING

Handeln von einer Familie $(S_i)_{i \in I}$ von nichtleeren endlichen Mengen. In diesen Problemen geht es um die Existenz einer Teilfamilie mit bestimmten Eigenschaften.

SET-PACKING

Wie viele der Mengen S_i kann man in U hineinpacken, ohne dass sie sich überlappen? Gibt es k Teilmengen in S , welche disjunkt sind und die Menge U abdecken?

Beispiel:

Medizinische Studie: 17 Allergene wählen, sodass jeder Proband maximal auf eines davon reagiert.

Lösung:

$I =$ Allergene, $S_i =$ auf Allergien i allergische Probanden, $J =$ Ausgewählte Allergene, $S_i \cap S_j = \emptyset \rightarrow$ Ausschlussbedingung zwischen Allergenen

SET-COVERING

Gibt es eine Unterfamilie bestehend aus k Mengen, die die gleiche Vereinigung U hat? Kann man k Teilmengen bilden, welche die Menge S komplett abdecken?

Beispiel:

Ein Tourist möchte mit dem Besuch von nur k Aussichtspunkten alles sehen, was man an Sehenswürdigkeiten von allen Aussichtspunkten aus sehen könnte.

Lösung:

$U =$ Alle Sehenswürdigkeiten, $i \in I =$ Aussichtspunkt i , $S_i =$ Sichtbare Sehenswürdigkeiten von i aus, $J \subset I =$ Ausgewählte Aussichtspunkte, $\bigcup_{i \in J} S_i = U =$ Bedingung, alles sehen zu können

EXACT-COVER

Gibt es eine Unterfamilie von Mengen, die disjunkt sind, aber die gleiche Vereinigung haben? Jedes Element in U soll genau in einer der Teilmengen der Familie S vorkommen. Die gesuchte Menge bildet eine exakte Überdeckung.

Beispiel:

Es stehen binäre Eigenschaften von Kunden zur Verfügung, z.B. ob sie ein bestimmtes Produkt gekauft haben oder volljährig sind. Ziel: Eine Teilmenge von Kriterien finden, dass jeder Kunde genau eine der Eigenschaften hat.

Lösung:

$I =$ Eigenschaften, $(S_i)_{i \in I} =$ Kunden, auf die die Eigenschaft zutrifft, $J \subset I =$ Teilmenge

HITTING-SET

Sucht zu U eine Menge von Punkten (Elemente), die jede Menge der Familie in genau einem Punkt treffen. Gibt es eine Menge $W \subset U$ derart, dass W mit jeder Menge S_i nur ein Element gemeinsam hat?

Beispiel:

Finden eines Einzelnen Vertreters für Gruppen von Menschen.

Lösung:

$i \in I =$ Gruppierungskriterien, $S_i =$ Menschen in Gruppe i , $W \subset \bigcup_{i \in I} S_i =$ Teilmenge von Vertretern, $|W \cap S_i| = 1 \forall i \rightarrow H$ hat mit jeder der Mengen S_i genau einen Vertreter gemeinsam

STEINER-TREE

Gibt es zu einem Graphen $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$, einer Teilmenge $R \subset V$ der Knoten und einer Zahl k einen Teilbaum $T = (V_1, E_1) \subset G$ des Graphen, der alle Knoten von R enthält und dessen Kantengewicht $\sum_{e \in E_1} \varphi(e) \leq k$ ist?

Beispiel:

Stromnetzausbau zwischen Ortschaften gegeben eines Budgets.

Lösung:

STEINER-TREE = Stromnetz, Knoten $V =$ Ortschaften, Knoten $R \subset V =$ zu erschliessende Ortschaften, Gewicht $\varphi =$ Baukosten einer Verbindung, Max. Gewicht $k =$ Budget

SEQUENCING

Die Komponenten der drei Vektoren

Laufzeiten:

$T = (T_1, \dots, T_p) \in \mathbb{Z}^p$

Deadlines:

$D = (D_1, \dots, D_p) \in \mathbb{Z}^p$

Strafen:

$P = (P_1, \dots, P_p) \in \mathbb{Z}^p$

Handeln von einer Familie $(S_i)_{i \in I}$ von nichtleeren endlichen Mengen. In diesen Problemen geht es um die Existenz einer Teilfamilie mit bestimmten Eigenschaften.

SET-PACKING

Wie viele der Mengen S_i kann man in U hineinpacken, ohne dass sie sich überlappen? Gibt es k Teilmengen in S , welche disjunkt sind und die Menge U abdecken?

Beispiel:

Medizinische Studie: 17 Allergene wählen, sodass jeder Proband maximal auf eines davon reagiert.

Lösung:

$I =$ Allergene, $S_i =$ auf Allergien i allergische Probanden, $J =$ Ausgewählte Allergene, $S_i \cap S_j = \emptyset \rightarrow$ Ausschlussbedingung zwischen Allergenen

SET-COVERING

Gibt es eine Unterfamilie bestehend aus k Mengen, die die gleiche Vereinigung U hat? Kann man k Teilmengen bilden, welche die Menge S komplett abdecken?

Beispiel:

Ein Tourist möchte mit dem Besuch von nur k Aussichtspunkten alles sehen, was man an Sehenswürdigkeiten von allen Aussichtspunkten aus sehen könnte.

Lösung:

$U =$ Alle Sehenswürdigkeiten, $i \in I =$ Aussichtspunkt i , $S_i =$ Sichtbare Sehenswürdigkeiten von i aus, $J \subset I =$ Ausgewählte Aussichtspunkte, $\bigcup_{i \in J} S_i = U =$ Bedingung, alles sehen zu können

EXACT-COVER

Gibt es eine Unterfamilie von Mengen, die disjunkt sind, aber die gleiche Vereinigung haben? Jedes Element in U soll genau in einer der Teilmengen der Familie S vorkommen. Die gesuchte Menge bildet eine exakte Überdeckung.

Beispiel:

Es stehen binäre Eigenschaften von Kunden zur Verfügung, z.B. ob sie ein bestimmtes Produkt gekauft haben oder volljährig sind. Ziel: Eine Teilmenge von Kriterien finden, dass jeder Kunde genau eine der Eigenschaften hat.

Lösung:

$I =$ Eigenschaften, $(S_i)_{i \in I} =$ Kunden, auf die die Eigenschaft zutrifft, $J \subset I =$ Teilmenge

HITTING-SET

Sucht zu U eine Menge von Punkten (Elemente), die jede Menge der Familie in genau einem Punkt treffen. Gibt es eine Menge $W \subset U$ derart, dass W mit jeder Menge S_i nur ein Element gemeinsam hat?

Beispiel:

Finden eines Einzelnen Vertreters für Gruppen von Menschen.

Lösung:

$i \in I =$ Gruppierungskriterien, $S_i =$ Menschen in Gruppe i , $W \subset \bigcup_{i \in I} S_i =$ Teilmenge von Vertretern, $|W \cap S_i| = 1 \forall i \rightarrow H$ hat mit jeder der Mengen S_i genau einen Vertreter gemeinsam

STEINER-TREE

Gibt es zu einem Graphen $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$, einer Teilmenge $R \subset V$ der Knoten und einer Zahl k einen Teilbaum $T = (V_1, E_1) \subset G$ des Graphen, der alle Knoten von R enthält und dessen Kantengewicht $\sum_{e \in E_1} \varphi(e) \leq k$ ist?

Beispiel:

Stromnetzausbau zwischen Ortschaften gegeben eines Budgets.

Lösung:

STEINER-TREE = Stromnetz, Knoten $V =$ Ortschaften, Knoten $R \subset V =$ zu erschliessende Ortschaften, Gewicht $\varphi =$ Baukosten einer Verbindung, Max. Gewicht $k =$ Budget

SEQUENCING

Die Komponenten der drei Vektoren

Laufzeiten:

$T = (T_1, \dots, T_p) \in \mathbb{Z}^p$

Deadlines:

$D = (D_1, \dots, D_p) \in \mathbb{Z}^p$

Strafen:

$P = (P_1, \dots, P_p) \in \mathbb{Z}^p$

Ein hamiltonscher Pfad in einem Graphen $G = (V, E)$ ist ein Pfad, der jeden Knoten des Graphen genau einmal besucht. Ein solcher Pfad definiert eine Reihenfolge $a_1, \dots, a_n$ von Knoten, so dass jede Kante von a_i nach a_{i+1} in E ist. Für einen gerichteten Graphen sind dies die Kanten $(a_i, a_{i+1}) \in E$, für einen ungerichteten Graphen gilt $\{a_i, a_{i+1}\} \in E$. Ein hamiltonscher Zyklus ist ein geschlossener hamiltonscher Pfad.

HAMPATH

Gibt es einen gerichteten hamiltonschen Pfad im gerichteten Graphen G ?

Beispiel:

Ganze Schweiz bereisen, ohne eine Stadt mehrmals zu besuchen

Lösung:

$G =$ Liniennetz, $V =$ Stadt, $E =$ ÖV-Verbindung, Hamiltonscher Pfad = Reise

HAMCIRCUIT

Wie HAMPATH, aber zusätzlich muss der Pfad geschlossen sein

UHAMPATH

Wie HAMPATH, aber ungerichtet

UHAMCIRCUIT

Wie UHAMPATH, aber zusätzlich muss der Pfad geschlossen sein

Es handelt sich um die Eigenschaften von Zyklen in gerichteten Graphen $G = (V, E)$. Die Kantenmenge E besteht aus gerichteten Kanten (a, e) mit $a, e \in V$. Ein gerichteter Zyklus in G ist eine Folge von Knoten $a_0, \dots, a_n = a_0$, die jeweils Enden von Kanten sind, also $(a_i, a_{i+1}) \in E, i = 0, \dots, n-1$

FEEDBACK-NODE-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $W \subset V$ von k Knoten derart, dass jeder geschlossene Zyklus durch einen dieser Knoten geht?

Beispiel:

Es gibt mehrere Buslinien. Wo muss das Putzpersonal platziert werden, damit alle Linien geputzt werden können? Man möchte möglichst wenig Personal einsetzen.

Lösung:

$V =$ Stationen, $W \subset V =$ Stationen, von denen aus alle Linien erreicht werden

FEEDBACK-ARC-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $F \subset E$ von k Kanten derart, dass jeder geschlossene Zyklus eine dieser Kanten enthält?

Gegeben ein Graph $G = (V, E)$ mit Knoten V und Kanten K und eine Zahl k . Ist es möglich, die Knoten des Graphen mit k Farben so einzufärben, dass durch Kanten verbundene Knoten verschiedene Farben haben?

Beispiel:

Stundenplan: Planung der Fächer $f \in F$ auf k Zeitfenster, damit keine Anmeldungen $\{f_1, f_2\} \in A$ in gleichen Zeitfenstern liegen

Lösung:

Knoten $V =$ Fach, Kanten $E =$ Anmeldungen, Farben = Zeitfenster

Gibt es k Cliques so, dass jede Ecke in genau einer der Cliques ist?

CLIQUE-COVER

Gibt es k Cliques so, dass jede Ecke in genau einer der Cliques ist?

Wie CLIQUE-COVER, nur ist k bekannt.

Beispiel:

Job-Parallelisierbarkeit: Menge von n Jobs, die jeweils exklusiv auf m Ressourcen zugreifen, Jobs dürfen nicht gleichzeitig laufen. Ziel: Entscheiden, ob zu irgendeinem Zeitpunkt mehr als k Prozessoren ausgelastet werden können

Lösung:

Graph $G =$ Job-Parallelisierbarkeit, Knoten $V = n$ Job, Kanten $E = m$ gleiche Ressourcen, k -Clique $G_k =$ Auswahl Knoten ohne Verbindung, $k =$ Auslastbare Anzahl Prozessoren

Gibt es eine Menge W von k Knoten derart, dass jede Kante von G einen Endpunkt in W hat?

Beispiel:

Kontrolle des Verkehrsnetzes: Mitarbeiter haben ihre Basis an einzelnen Knotenpunkten. Ziel: An welchen Stationen muss man Kontrolleure stationieren, damit jede Strecke bei einem Kontrolleur endet?

Lösung:

Knoten $V =$ Stationen, Kanten $E =$ Strecke, $k =$ Anzahl Kontrolleure, $W \subset V =$ Knoten mit Kontrolleur

Gegeben eine aussagenlogische Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

Beispiel:

Elektriker: Konfiguration der Schalter zweier Stromkreise, die in allen Räumen Licht brennen lassen sollten

Lösung:

$x_i =$ Schalter, $T \vee F =$ Stromkreise (werte der Variable x_i , Lampen des ersten Kreises leuchten, wenn $x_i = T$), $x_1 \vee \dots \vee x_6 =$ ob Licht im Raum der mit x_i verbundenen Schalter brennt

3SAT

Wie SAT, nur hat jede Klausel der Normalform maximal 3 Literale.

Beispiel:

SAT $\rightarrow$ 3SAT

Lösung:

$(x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_5 \vee x_6) \rightarrow (x_1 \vee x_2 \vee x_1) \wedge (\overline{x_1} \vee x_3 \vee x_4) \wedge (x_5 \vee x_6 \vee x_6)$

Gegeben eine Liste S von natürlichen Zahlen $s_i \in \mathbb{N}$ und einer natürlichen Zahl $t \in \mathbb{N}$, ist es möglich eine Teilliste $T \subset S$ auszuwählen, sodass die Elemente von T die Summe $t = \sum_{s \in T} s$ haben?

Beispiel:

Rucksack-Problem (Ziel: Rucksack füllen)

Lösung:

$S =$ Grössen der Gegenstände, $t =$ Max. Platz im Rucksack, $T \subset S =$ Grössen der Gegenstände, die in den Rucksack passen

3D-MATCHING

Gegeben sind drei endliche Mengen X, Y und Z mit gleich vielen $n = |X| = |Y| = |Z|$ Elementen und $T \subset X \times Y \times Z$. Gibt es eine n -elementige Teilmenge $M \subset T$ derart, dass keine zwei Tripel von M in einer Koordinate übereinstimmen?

Beispiel:

Ein Koch hat n Rezepte für Vorspeisen, Hauptspeisen und Desserts. Nicht alle Vorspeisen lassen sich mit jeder Hauptspeise kombinieren, dasselbe gilt auch für Desserts. Er möchte n Menus zusammenstellen, sodass jedes Rezept in genau einem der Menus vorkommt.

Lösung:

$X =$ Vorspeisen, $Y =$ Hauptspeisen, $Z =$ Desserts, $T \subset X \times Y \times Z =$ Menge aller Menus, $M \subset T =$ Menge der zusammengestellter Menus

BIP

BIP = Binary Integer Programming

Zu einer ganzzahligen Matrix C und einem ganzzahligen Vektor d , ist ein binärer Vektor x zu finden mit $C \cdot x = d$

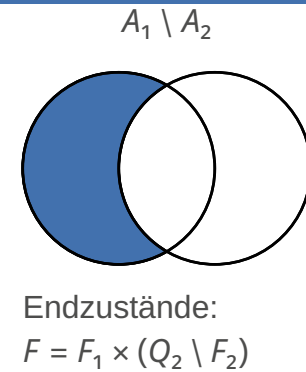
Beispiel:

$C = \begin{pmatrix} 1 & 3 & 0 \\ 0 & 2 & 5 \end{pmatrix} \quad d = \begin{pmatrix} 1 \\ 5 \end{pmatrix}$
 $x = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$ da $\begin{pmatrix} 1 & 3 & 0 \\ 0 & 2 & 5 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 5 \end{pmatrix}$

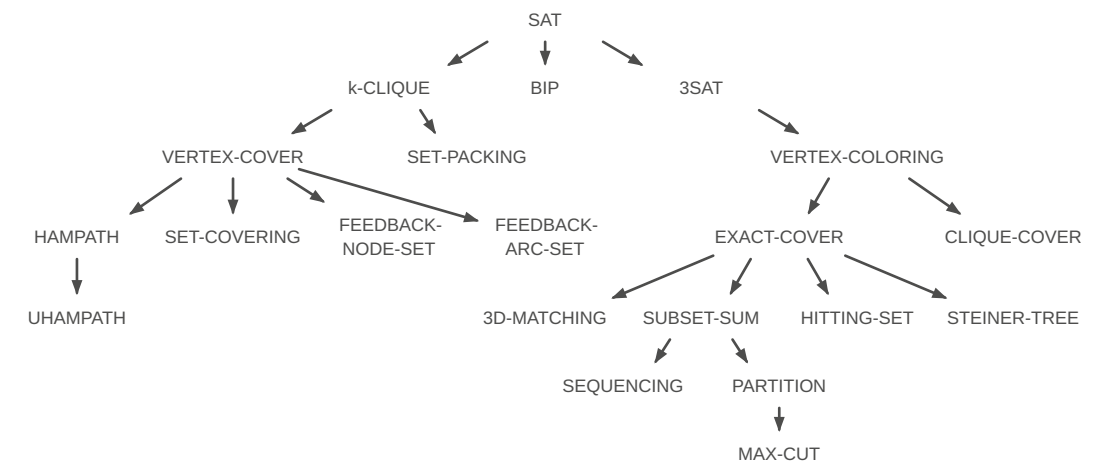
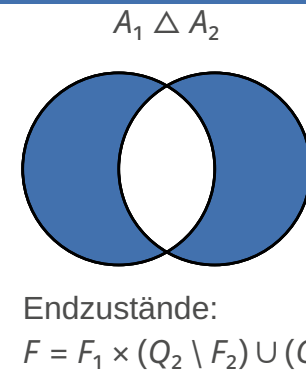
AutoSpr | FS26

Seite 4

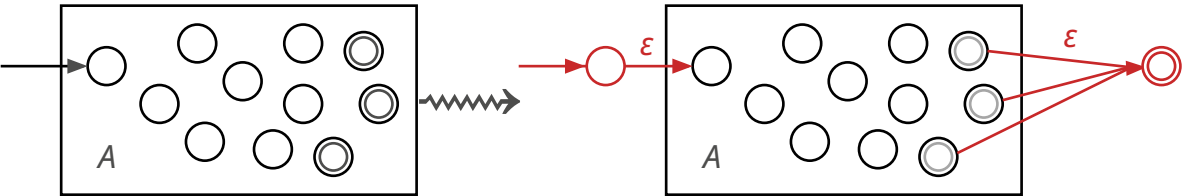
DIFFERENZMENGE $L(A_1) \setminus L(A_2)$



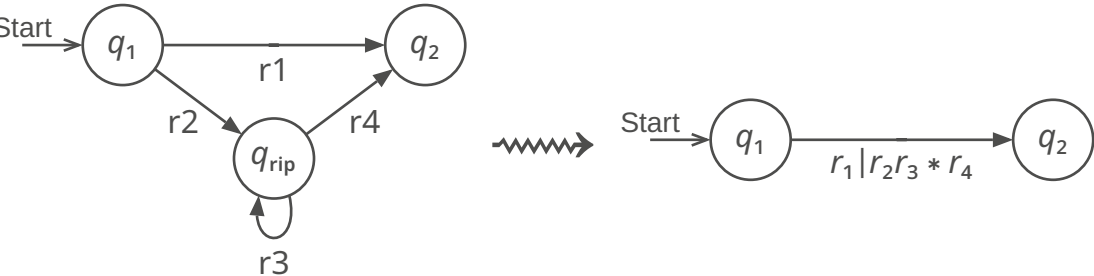
SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$



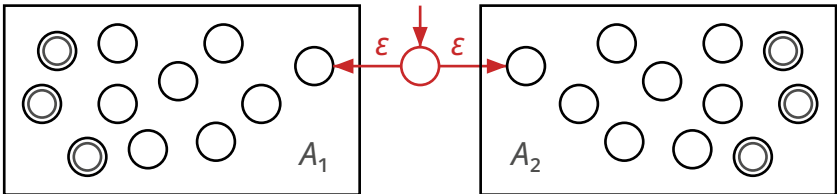
Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A

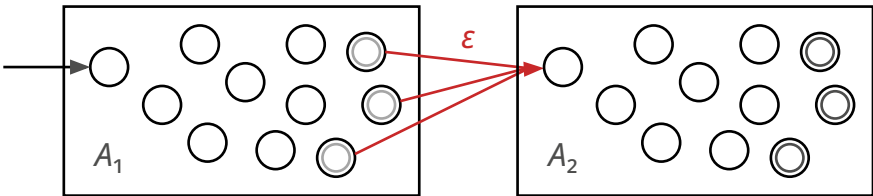


$$L = L_1 \cup L_2$$
$$= L(A_1) \cup L(A_2)$$
$$= L(A_1) \mid L(A_2)$$



VERKETTUNG

$$L = L_1 L_2$$
$$= L(A_1) L(A_2)$$
$$= \{w_1 w_2 \mid w_i \in L_i\}$$



*-OPERATION

$$L^* = \{\epsilon\} \cup L \cup L^2 \cup \dots$$
$$= \bigcup_{k=0}^{\infty} L^k$$

